

Solution for Water Slides

The problem asks us to maximize the total quality of the slides, which is the sum of their individual height differences.

Suppose we have a set of slides such that the i -th of them leads from s_i to t_i . The value we're optimizing is $\sum_{i=1}^k (H[s_i] - H[t_i])$.

The first key observation is that we can rewrite this sum as follows: $\left(\sum_{i=1}^k H[s_i]\right) - \left(\sum_{i=1}^k H[t_i]\right)$.

To maximize the value of this formula, we want to make the first part as large as possible and the second part as small as possible. Therefore, we clearly cannot do any better than taking the highest k points as the starts and the lowest k points as the ends.

At this moment, we don't know yet whether such a solution is really feasible. But at least we can easily tell that:

- As $2k \leq n$, the two sets of points are disjoint.
- As all heights are distinct, the height of each start is strictly bigger than the height of each end. I.e., if we connect any start to any end, we will always get a slide that goes downhill.

Avoiding crossings

We now claim that the theoretically optimal solution described above can always be built.

What we have are k starting points and k ending points arranged in some order around a circle. Our goal is to check whether we can pair the starts to ends in some way that can be drawn with no crossings.

We can construct one such pairing efficiently using a greedy stack-based approach.

We will have two stacks: `unmatched_starts` and `unmatched_ends`. Initially, both are empty. During the algorithm, at most one of them will be non-empty at any time.

We will walk around the island once and do the following:

- Whenever we encounter a starting point: if we have some `unmatched_ends`, we pop the most recently inserted one and pair it with the current start. Otherwise, push the current start into `unmatched_starts`.
- Whenever we encounter an ending point: we do the symmetric thing — i.e., pair it with the last unmatched start if we have one or push it into the stack of unmatched ends if we don't have any pending start.

It should be obvious that this algorithm always matches the starts with ends somehow. (By induction, at any moment the number of unmatched points waiting in a stack corresponds to the difference between the counts of processed starts and ends up to that point. Thus, at the end both stacks must be empty, which means that each start got matched with some end.)

And we can also easily verify that the slides that correspond to the constructed matching can be drawn without intersections. (Pick and draw any one slide $s \rightarrow t$. Divide all other starts and ends into two piles. The first pile will be the starts and ends we processed while one of s and t waited in a stack for its partner. The other pile will be all other starts and ends. Clearly, for each point, its partner is in the same pile, so their slide doesn't need to cross the $s \rightarrow t$ slide. Continue recursively separately with each pile until we get a full valid drawing.)

It is possible to implement the full solution so that it runs in linear time (using linear-time k -th element to find the starts and ends), but given the nature of the contest, why bother, right? Hence, the reference solution simply sorts the points and runs in $O(n \log n)$. The difference is negligible.